



TSSB-3M: Mining single statement bugs at massive scale

Cedric Richter*
University of Oldenburg

Oldenburg, Germany
cedric.richter@uni-oldenburg.de

Heike Wehrheim*
University of Oldenburg

Oldenburg, Germany
heike.wehrheim@uni-oldenburg.de

ABSTRACT

Single statement bugs are one of the most important ingredients in the evaluation of modern bug detection and automatic program repair methods. By affecting only a single statement, single statement bugs represent a type of bug often overlooked by developers, while still being small enough to be detected and fixed by automatic methods. With the rise of data-driven automatic repair the availability of single statement bugs at the scale of millionth of examples is more important than ever; not only for testing these methods but also for providing sufficient real world examples for training. To provide access to bug fix datasets of this scale, we are releasing two datasets called SSB-9M and TSSB-3M. While SSB-9M provides access to a collection of over 9M general single statement bug fixes from over 500K open source Python projects, TSSB-3M focuses on over 3M single statement bugs which can be fixed solely by a single statement change. To facilitate future research and empirical investigations, we annotated each bug fix with one of 20 single statement bug (SStuB) patterns typical for Python together with a characterization of the code change as a sequence of AST modifications. Our initial investigation shows that at least 40% of all single statement bug fixes mined fit at least one SStuB pattern, and that the majority of 72% of all bugs can be fixed with the same syntactic modifications as needed for fixing SStuBs.

KEYWORDS

Datasets, single statement bugs, Python, open software repositories

ACM Reference Format:

Cedric Richter and Heike Wehrheim. 2022. TSSB-3M: Mining single statement bugs at massive scale. In *19th International Conference on Mining Software Repositories (MSR '22)*, May 23–24, 2022, Pittsburgh, PA, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3524842.3528505>

1 INTRODUCTION

Software bugs, i.e. unintended behavior introduced by a programmer mistake, come in many forms ranging from small errors affecting only a single token to major design flaws requiring complete program rewrites. Especially small bugs in a single line or statement only can be easily overlooked by a developer. For this

*This author was partially supported by the German Research Foundation (DFG) within the Collaborative Research Centre "On-The-Fly Computing" (SFB 901).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MSR '22, May 23–24, 2022, Pittsburgh, PA, USA

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9303-4/22/05...\$15.00

<https://doi.org/10.1145/3524842.3528505>

reason and to relieve the developer from the burden of manually localizing and repairing small, simple bugs, several works have explored automatic methods for bug identification [1, 8, 16–19] and program repair [5, 14, 22, 23]. While traditional methods [2, 20] often rely on hand-crafted rules to identify potential bugs, recent methods [16, 17, 23] started to explore data-driven techniques. Such techniques, however, require access to huge databases of known bugs for training. Collecting single statement bugs at the required scale is difficult, though, as they only rarely occur in open source projects. Therefore, existing methods often had to rely on generated *pseudo bugs* for producing a sufficient amount of training data.

In this paper, we address the lack of available training examples (for Python) by releasing two new ultralarge collections of single statement bug fixes. For this, we mined over 500K Python open source projects for bug-fixing commits that only change a single statement. By following a mining process similar to previous work [12], we derived a dataset of more than 9M bug fixes (SSB-9M). Since the collected bug fixes might be tangled with unrelated code changes [9], we additionally filtered our dataset for code changes that fully patch a bug with a single statement change - without modifying source code in other files or statements in the same file at a different location. This more restrictive process produced a collection of over 3M "true" single statement bugs (TSSB-3M). To compare the distribution of bugs in our datasets with previous such collections like PySStuBs for Python [11], we annotate each bug fix with one of 20 simple single statement bug (SStuB) patterns. In total, we find that around 40% of all mined bugs can be assigned to at least one pattern. To also explore the remaining bugs, we additionally annotate each bug with an *AST edit script*. Using four edit operation types, a script describes how the AST of the buggy code has to be changed to arrive at its repaired version. Based on the computed edit script, we find that the majority of single statement bugs (72% of all collected bugs) can be fixed with the same edit operation needed to fix a SStuB. Still, there exists a small percentage (2% of all collected bugs) which are not even remotely similar to bugs matching a SStuB pattern and, hence, require special care when addressed with automatic repairing methods. We hope that a collection of bug fixes at this scale can not only facilitate future research in data-driven bug detection and repairing methods, but can also shed light on what types of simple single statement bugs developers introduce in code and how they fix them.

2 METHODOLOGY

In the following, we provide a detailed overview over our methodology and the individual steps necessary for building the datasets.

2.1 Identifying Appropriate Python Projects

In order to find suitable repositories for mining single statement bugs we employ the Libraries.io 1.6 [13] package index. The package

index includes over 2 million references to Python Git repositories which we employ as a starting point for our mining process. Although mining on popular projects ensures a well-maintained code base, we intentionally decided against excluding repositories based on their popularity. We believe that all bug fixes are equally valuable for gaining insights. Repositories of lower popularity might provide access to bugs which typically occur during the development, but would be caught before arriving in a better maintained project. Furthermore, we also include fork projects since they might provide access to bugs not occurring in the original project. To avoid commit duplication, we later remove all duplicate commits. Finally, note that our datasets do not include commits from all projects listed in the index since not all projects are publicly accessible or contain a single statement commit.

2.2 Mining Single Line Edits

To effectively mine single statement bugs at the scale of millionth of repositories, we start by determining potential bug *candidates* with a cheap approximative process. Instead of identifying single statement bugs directly, we first crawl the repositories for commits that modify exactly one line per modified file. During this process, we ignore all changes to the formatting and comments since they do not alter the program behavior. Checking whether a commit contains a single line edit can be performed at a textual level without requiring to parse the complete abstract syntax tree (AST) of the program. In addition, lines in Python generally correspond to single statements by design. Even if a statement is defined over multiple lines, we still capture single statement modifications that are only performed at a single location. However, we do not capture modifications to a single multi-line statement at multiple locations. To finally determine whether a commit contains a single line edit, we compute the difference between the program before and after the patch. Similar to the Unix diff algorithm, we consider all file modifications as removing old lines and adding the modified lines. To identify single line changes, we count how many lines have been modified by comparing removed and added lines. For the comparison, we tokenize each line and compare the respective token sequences. By employing a tokenizer¹, we automatically ignore any changes to the code formatting. In addition, our tokenizer is configured to ignore all changes to comments since they do not alter the program behavior. Finally, we store all commits that modify exactly one line together with all computed file differences.

For the mining process, the workload is distributed on a cluster of over 1000 workers. Each worker iteratively clones the assigned projects and searches through the commit history for single line commits. During this process, we ignore all commits that either add or remove complete files, since we are only interested in single code changes. The complete mining process took around two weeks and produced a total of over 66M single line edits from more than 500K git projects.

2.3 Selecting Single Statement Changes

After collecting single line edits from thousands of projects, we were interested in finding all *single statement* changes. For this, we iterate over all collected commits, while analyzing the previously stored file

¹For tokenization, we employ the tokenize package.

difference for single statement changes. We exclude all commits that (1) are duplicates introduced by fork projects, (2) do not contain a full parsable statement or (3) change multiple statements in a single line. For identifying whether a commit changes a statement, we compute the AST for the file difference. It usually contains changed lines in addition to some context code lines. Although the code in a file difference is generally not enough to build the full AST, it is enough for the purpose of single statement change identification. We employ a best-effort AST parser² to compute the two ASTs for the code line before and after the fix. To locate the difference in the AST representation, we perform a simultaneous depth-first search, similar to Kampatsis and Sutton [12], until we find the first node where the two ASTs differ. We exclude all commits where the computed AST node is not located inside a statement or is a root for multiple statements.

After filtering, we now remain with over 28M single statement changes from over 460K Git projects. Note that already by deduplication the size of the dataset nearly halves to over 33M single line edits. Since we believe that this dataset of single statement changes can be valuable for future research on the evolution of software projects, we publish it under the name SSC-28M. However, our main focus is still on the study of real single statement bugs.

2.4 Identifying True Single Statement Bugs

In this work, we consider two types of bug-fixing commits: *Single Statement Bugs*, which are obtained following the procedure of previous works, and *True Single Statement Bugs*, which are determined with a more restrictive but also more precise selection procedure. First of all, to determine whether a commit is bug-fixing, we check its commit message for the occurrence of at least one of the following keywords: 'error', 'bug', 'fix', 'issue', 'mistake', 'incorrect', 'fault', 'defect', 'flaw', and 'type'. This heuristics was repeatedly shown to be highly precise [5, 11, 12, 22] (with an accuracy of at least 90%). In addition, this is a common procedure to filter for bug-fixing commits when the created datasets are too large to be manually inspected [12, 22].

While this heuristics is effective to identify bug-fixing commits, it assumes that commit message and code change are related. However, as shown by prior work [9], a significant portion of bug fixing commits are entangled commits and, hence, change more code than it is necessary to fix a bug. Therefore, as soon as we look at the commit not as a whole, but only a part of it (in our case a single statement change), we risk that the change is unrelated to the bug fix. This motivates our second bug category of True Single Statement Bugs (TSSBs). TSSBs refer to bug fixing commits that fully patch a bug with exactly one single statement change. This is not only a helpful property for the evaluation of bug detection methods³, but also avoids entangled commits by definition. In addition, we do not apply commit unrolling [11, 12], which would split a single bug-fixing commit into multiple partial fixes. Now, after filtering the dataset for single statement bugs and true single statement bugs, we obtain two datasets SSB-9M and TSSB-3M, which contain over 9M and 3M bug fixes, respectively.

²To parse partial code, we employ the tree-sitter library.

³A common assumption in the evaluation of bug detection methods is that the code before the patch is buggy and correct after the patch. This is generally not true for commits belonging to traditional single statement bugs.

2.5 Characterizing Bug Patching Edits

For analyzing the types of bugs collected in our datasets, we employ two ways for characterizing a bug fix: (1) SStuB patterns and (2) AST Edits. SStuB patterns [12] are used to categorize bug fixes into frequently occurring bug type categories like changes in the usage of function names or binary operators. To categorize the collected bug fixes, we assign each commit to a unique bug pattern. Similar to [11], we assign each bug to the most specific category when it is matched by more than one SStuB pattern. In general, we distinguish 20 typical SStuB patterns as identified for Python and described by previous work [11]. If a commit does not fit a bug pattern, we assign it to a generic *single statement* pattern or *single token* pattern (when the commit only modifies a single token). In total, we found 50% to 60% of all instances in our datasets to not fit a single SStuB pattern.

To better characterize how developers make single statement mistakes, we further analyze the AST edits between the code before and after the code change. An AST edit script [6] describes how the AST of the buggy code has to be transformed to arrive at its fixed version. Every edit script consists of four types of AST operations: INSERT (inserting a new AST node at a given location), MOVE (moving an existing AST node to another position in the same AST), UPDATE (updating the value of a single node) and DELETE (removing a single node). For computing the AST difference, we employ the same two ASTs employed to filter the datasets and computed from the file difference. Then, we compute for each bug patch an AST edit script by employing a reimplementation of the GumTree algorithm [6]. By characterizing a bug patch through an AST edit script, we are able to analyze which operations have to be supported by a bug repairing method to fix a given bug. Overall, we found that single statement bugs can be fixed on average with 4.20 AST operations in SSB-9M and 4.52 AST operations in TSSB-3M.

2.6 Datasets

As a result of our mining process, we are releasing three new datasets: (1) TSSB-3M, a dataset of over 3M isolated single statement bug fixes, (2) SSB-9M, a dataset of over 9M general single statement bugs and (3) SSC-28M, a dataset of over 28M general single statement changes. Our intention is to facilitate future research on large scale datasets in software evolution, data-driven bug detection or repair. All collected datasets are novel and were not used in previous studies. To store millionth of bug fixes, all mined data is stored in a compressed jsonlines⁴ format. Every dataset entry contains information about (1) the project and commit hash of the bug fix, (2) the file difference as a Unix diff between the code before the change and after and (3) additional analytical results such as the identified SStuB pattern and AST edit script. Because of concerns regarding licensing, we cannot redistribute the complete code related to a bug fix. However, source code is available in the original projects and can be referenced via our datasets. All released datasets are publicly available on Zenodo⁵ and our tool for generating the datasets can be found on Github⁶ and Zenodo⁷.

⁴<https://jsonlines.org>

⁵<https://doi.org/10.5281/zenodo.5845439>

⁶<https://github.com/cedricrupb/TSSB3M>

⁷<https://doi.org/10.5281/zenodo.5898547>

Table 1: SStuB pattern statistics for SSB-9M, TSSB-3M and PySStuBs

SStuB Pattern	TSSB-3M		SSB-9M		PySStuBs [11]	
	Count	%	Count	%	Count	%
Change Identifier Used	237K	16	659K	18	9K	12
Change Binary Operand	174K	12	349K	9	4K	6
Same Function More Args	150K	10	457K	12	10K	14
Wrong Function Name	134K	9	397K	11	9K	12
Add Function Around Expression	117K	8	244K	6	6K	9
Change Attribute Used	104K	7	285K	8	5K	7
Change Numeric Literal	97K	6	275K	7	5K	7
More Specific If	68K	5	121K	3	2K	3
Add Method Call	60K	4	118K	3	3K	5
Add Elements To Iterable	57K	4	175K	5	2.5K	3
Same Function Less Args	50K	3	169K	4	3.4K	5
Change Boolean Literal	37K	3	82K	2	1.5K	2
Add Attribute Access	32K	2	74K	2	1.5K	2
Change Binary Operator	29K	2	71K	2	<1K	1
Same Function Wrong Caller	25K	2	46K	1	1.2K	2
Less Specific If	22K	2	45K	1	<1K	1
Change Keyword Argument Used	20K	1	59K	2	1.5K	2
Change Unary Operator	15K	1	23K	<1	2K	3
Same Function Swap Args	8K	<1	77K	2	<1K	1
Change Constant Type	6K	<1	12K	<1	2K	3
NoSStuB - Single Statement	1.15M	-	3.37M	-	-	-
NoSStuB - Single Token	740K	-	2.20M	-	-	-
Total SStuBs	1.46M	100	3.74M	100	73K	100
Total	3.34M	-	9.34M	-	-	-

3 ANALYSIS OF DATASETS

Our main contribution is a collection of bugs a magnitude larger than any existing bug dataset. Nevertheless, we also explored the datasets to gain new insights into developer bugs and their fixes.

Specifically, we were interested in the following two questions:

RQ1 Do we find the same distribution of SStuBs in our datasets as in previously collected datasets?

RQ2 How different are general single statement bugs from those selected by SStuB patterns?

RQ1. We measure the frequency of SStuBs in datasets TSSB-3M, SSB-9M and a previously explored Python SStuB dataset called PySStuBs [11]. The number of occurrences of each pattern in each dataset is given in Table 1. In general, we find that the distribution of SStuBs changes between datasets while the *ranking* of pattern frequencies is highly correlated (with a Spearman rank correlation of 0.94 between TSSB-3M and SSB-9M and 0.86 between TSSB-3M and PySStuBs). In particular, we observe that patterns such as *Change Identifier Used* and *Same Function More Args* are highly frequent in all datasets. Interestingly enough, some bug categories such as *Change Binary Operand* become more dominant in the TSSB-3M dataset. This could indicate that these types of bugs are harder to identify by a developer and, hence, are more likely to be fixed after the main development phase in an independent fix. Finally, we find that *only 40% - 43% of all Python single statement bugs* fit a SStuB pattern. While this finding is inline with previous observations on Java SStuBs [12], we are interested in the types of bugs occurring in the remaining dataset which is what we study in RQ2.

RQ2. We measure the similarity between single statement bugs that do not fit a SStuB pattern (NonSStuBs) and those that classify as a SStuB in the TSSB-3M dataset. The similarity between bug types

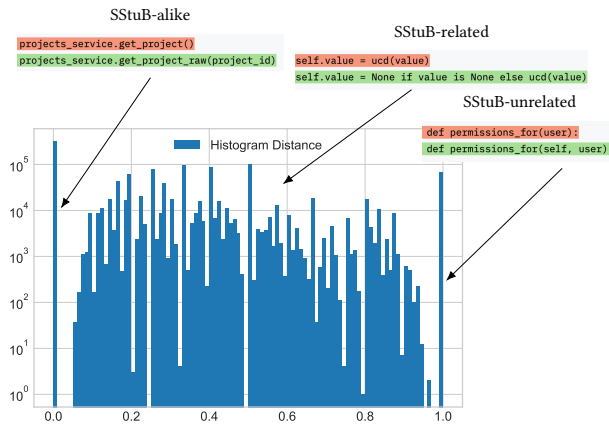


Figure 1: Similarity of NonSStuBs with SStuBs given as the Jaccard distance between edit operations. The x-axis corresponds to the binned distance to the most similar SStuB bug and the y-axis corresponds to observed frequencies.

is computed by the Jaccard distance [10] between the edit scripts of the respective bug fixes. Therefore, two bug fixes are similar if they share the same edit operations⁸. For the comparison, we always look at the minimal distances between NonSStuB and SStuB pairs. Figure 1 reports on the distribution of computed distances. The distance allows us to categorize NonSStuB bugs into three classes: SStuB-alike (distance of 0), SStuB-related (distance of around 0.5) and SStuB-unrelated (distance of 1). We find that SStuBs + SStuB-alike bugs populate around 72% of the dataset and can therefore be repaired by the same edit operations. Another frequent class are SStuB-related bugs, which are related to existing SStuB patterns but do not classify as a SStuB. For example, bug fixes that operate on inline if-conditions. While being if statement related, they are not covered by any actual SStuB pattern. Finally, we find that around 2% of all bugs (SStuB-unrelated) are completely unrelated to any SStuB pattern. Those types of bugs are often highly related to the specifics of the Python language. For example, we observed that developers commonly forget to add the implicit `self` argument to a Python method. The fact that this class of bugs has, however, a low frequency is quite promising for the research on bug detection and repairing methods: *Methods that supports the detection and repair of SStuBs are also likely well suited to identify and fix a wide range of the most common single statement bug types.*

4 RELATED WORK

Single statement bugs, especially SStuBs, were explored in previous works, not only as a data source [4, 11, 12] but also for methods that can detect and prevent this simpler type of bugs [5, 14, 15, 22]. Karampatsis and Sutton [12] collected around 63 thousand Java single statement bugs from 1000 popular projects that fit at least one of 16 SStuB categories. They found out that around 33% of

⁸Instead of the concrete edit operation that is highly specific to the modified AST we employ the type of an edit operation composed of operation name and types of its arguments. For example, the type of an operation that adds a binary operator to an assignment is `Insert(binary_op, assign)`.

all mined bugs can be classified as a SStuB. Kamienski et al. [11] explored similar types of bug fixes for the 1000 most popular Python projects⁹ while introducing 7 new SStuB patterns typical for Python. Their collection of bugs is of similar size with around 73 thousand examples. Similar to Kamienski et al. we also explored bugs that fit SStuB patterns in Python. However, our mining process allowed us to go beyond the most popular Python projects, which resulted in a dataset more than 20x larger than all existing SStuB collections. In addition and in contrast to previous bug collections, we also analyze edit operations needed to fix a bug. Therefore, we were able to explore bug types that are not covered by SStuB patterns. Tufano et al. [22] collected a set of 787 thousand bug-fixing single line commits in Java for machine-learning based program repair. They used the same heuristics we employed for identifying bug-fixing code changes. In addition, their method is trained to translate buggy code lines into its fixed version. Finally, Bader et al. [3] showed that AST edit scripts can be effectively employed for automatic program repair by predicting bug fixes based on previously seen AST transformations. Not only does our dataset provide more than 11x more training data (SSB-9M), which has the potential to improve the performance of existing methods [7, 21], but also by annotating each bug fix with an AST script our dataset can directly be employed in various training setups.

5 LIMITATIONS - THREATS TO VALIDITY

Even though the employed heuristics for identifying bug fixing commits has been repeatedly shown to be highly precise, there is still a chance for false positives in our datasets. To mitigate this problem, we have designed our mining process to be as precise as possible by avoiding commit unrolls and by filtering for isolated bug fixes in TSSB-3M. The latter not only guarantees that the causal relationship between commit message indicating a bug fix and code change persists, but also avoids the mining of entangled code changes. While we mined a large portion of all available open source projects, we were restricted to projects accessible to the public. The distribution of simple bugs in closed source projects or projects without version control might be different. Finally, while the mining process is general enough to be applied to other languages, our datasets are restricted to single statement bugs in Python. The concrete instantiations and frequency of bugs might vary for other programming languages and projects.

6 CONCLUSION

In this work, we explored a new mining process for single statement bugs which enabled us to crawl through the commits of more than 500K public git repositories. As a result of our mining process, we are releasing two new datasets of single statement bug fixes that are a magnitude larger than any existing collection of bug fixes. More precisely, we introduce SSB-9M, a dataset consisting of over 9M single statement changes in Python, as well as the 3M bug fix dataset TSSB-3M. TSSB-3M guarantees that every collected bug can be fixed by modifying only a single statement. We hope that datasets of this size will facilitate future research in data-driven bug detection and automatic program repair.

⁹Since our dataset mining is more recent and some of the projects mined in PySStuBs might not be public any longer, our datasets are not necessarily supersets of PySStuBs.

ACKNOWLEDGMENTS

The authors gratefully acknowledge the funding of this project by computing time provided by the Paderborn Center for Parallel Computing (PC²).

REFERENCES

- [1] Miltiadis Allamanis, Marc Brockschmidt, and Mahmoud Khademi. 2018. Learning to Represent Programs with Graphs. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net. <https://openreview.net/forum?id=BJOFETxR->
- [2] Nathaniel Ayewah, William Pugh, David Hovemeyer, J David Morgenthaler, and John Penix. 2008. Using static analysis to find bugs. *IEEE software* 25, 5 (2008), 22–29.
- [3] Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. 2019. Getafix: Learning to fix bugs automatically. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–27.
- [4] Dirk Beyer, Lars Grunske, Thomas Lemberger, and Minxing Tang. 2021. Towards a Benchmark Set for Program Repair Based on Partial Fixes. *CoRR* abs/2107.08038 (2021). [arXiv:2107.08038](https://arxiv.org/abs/2107.08038) <https://arxiv.org/abs/2107.08038>
- [5] Zimin Chen, Steve James Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. 2019. Sequencer: Sequence-to-sequence learning for end-to-end program repair. *IEEE Transactions on Software Engineering* (2019).
- [6] Jean-Rémy Falleri, Floréal Morandat, Xavier Blanc, Matias Martinez, and Martin Monperrus. 2014. Fine-grained and accurate source code differencing. In *ACM/IEEE International Conference on Automated Software Engineering, ASE '14, Vasteras, Sweden - September 15 - 19, 2014*. 313–324. <https://doi.org/10.1145/2642937.2642982>
- [7] Alon Halevy, Peter Norvig, and Fernando Pereira. 2009. The unreasonable effectiveness of data. *IEEE Intelligent Systems* 24, 2 (2009), 8–12.
- [8] Vincent J Hellendoorn, Charles Sutton, Rishabh Singh, Petros Maniatis, and David Bieber. 2019. Global relational models of source code. In *International conference on learning representations*.
- [9] Kim Herzig and Andreas Zeller. 2013. The impact of tangled code changes. In *2013 10th Working Conference on Mining Software Repositories (MSR)*. IEEE, 121–130.
- [10] Paul Jaccard. 1901. Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bull Soc Vaudoise Sci Nat* 37 (1901), 547–579.
- [11] Arthur V Kamienski, Luisa Palechor, Cor-Paul Bezemer, and Abram Hindle. 2021. PySStuBs: Characterizing Single-Statement Bugs in Popular Open-Source Python Projects. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. IEEE, 520–524.
- [12] Rafael-Michael Karampatsis and Charles Sutton. 2020. How often do single-statement bugs occur? the manysstubs4j dataset. In *Proceedings of the 17th International Conference on Mining Software Repositories*. 573–577.
- [13] Jeremy Katz. 2020. Libraries.io Open Source Repository and Dependency Meta-data. <https://doi.org/10.5281/zenodo.3626071>
- [14] Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. Coconut: combining context-aware neural translation models using ensemble for program repair. In *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*. 101–114.
- [15] Ehsan Mashhadi and Hadi Hemmati. 2021. Applying CodeBERT for Automated Program Repair of Java Simple Bugs. In *18th IEEE/ACM International Conference on Mining Software Repositories, MSR 2021, Madrid, Spain, May 17-19, 2021*. IEEE, 505–509. <https://doi.org/10.1109/MSR52588.2021.00063>
- [16] Michael Pradel and Satish Chandra. 2022. Neural software analysis. *Commun. ACM* 65, 1 (2022), 86–96. <https://doi.org/10.1145/3460348>
- [17] Michael Pradel and Koushik Sen. 2018. Deepbugs: A learning approach to name-based bug detection. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018), 1–25.
- [18] Cedric Richter and Heike Wehrheim. 2021. DeepMutants: Training neural bug detectors with contextual mutations. *arXiv preprint arXiv:2107.06657* (2021).
- [19] Cedric Richter and Heike Wehrheim. 2022. Learning Realistic Mutations: Bug Creation for Neural Bug Detectors. *15th IEEE International Conference on Software Testing, Verification and Validation* (2022), to be published.
- [20] Caitlin Sadowski, Jeffrey Van Gogh, Ciera Jaspan, Emma Soderberg, and Collin Winter. 2015. Tricorder: Building a program analysis ecosystem. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 1. IEEE, 598–608.
- [21] Chen Sun, Abhinav Shrivastava, Saurabh Singh, and Abhinav Gupta. 2017. Re-visiting unreasonable effectiveness of data in deep learning era. In *Proceedings of the IEEE international conference on computer vision*. 843–852.
- [22] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 28, 4 (2019), 1–29.
- [23] Marko Vasic, Aditya Kanade, Petros Maniatis, David Bieber, and Rishabh Singh. 2019. Neural Program Repair by Jointly Learning to Localize and Repair. *CoRR* abs/1904.01720 (2019). [arXiv:1904.01720](https://arxiv.org/abs/1904.01720) <http://arxiv.org/abs/1904.01720>